

Task 'Unit 11 Dependency Injection and Inversion of Control (IoC)' Unit 11

Dependency Injection (DI) and Inversion of Control (IoC) improvement

Dependency Injection (DI) and Inversion of Control (IoC) improve maintainability by introducing loose coupling between classes, in the original design, `UserManager` was directly instantiated `EmailService`, making it complicated to change the notification methods or perform isolated testing. Refactoring introduces the `NotificationService` abstraction, so dependencies are injected externally, supporting the Dependency Inversion Principle and improving flexibility.

```
```python
```

```
from abc import ABC, abstractmethod
```

```
class NotificationService(ABC):
```

```
 @abstractmethod
```

```
 def send(self, user, msg): pass
```

```
class EmailService(NotificationService):
```

```
 def send(self, user, msg):
```

```
 print(f"Email to {user}: {msg}")
```

```
class SMSService(NotificationService):
```

## Task 'Unit 11 Dependency Injection and Inversion of Control (IoC)' Unit 11

```
def send(self, user, msg):

 print(f"SMS to {user}: {msg}")
```

```
class UserManager:
```

```
 def __init__(self, notifier):

 self.notifier = notifier
```

```
 def register_user(self, user):

 self.notifier.send(user, "Welcome!")
```

```
class MockService(NotificationService):
```

```
 def __init__(self):

 self.called = False
```

```
 def send(self, user, msg):

 self.called = True
```

```
UserManager(EmailService()).register_user("Ali")
```

```
UserManager(SMSService()).register_user("Sara")
```

## Task 'Unit 11 Dependency Injection and Inversion of Control (IoC)' Unit 11

```
mock = MockService()

UserManager(mock).register_user("Test")

assert mock.called

'''
```

The SMSService demonstrates the new notification methods and how it can be added without modification of the UserManager, additionally, the mock object enables isolated unit testing without sending real messages. DI containers such as dependency-injector can automate dependency wiring in larger systems.

### References

- ETS Labs (2026) Dependency Injector documentation. Available at: <https://python-dependency-injector.ets-labs.org/> [Accessed: June 10, 2026].
- ETS Labs (2026) Dependency injection and inversion of control in Python. Available at: [https://python-dependency-injector.ets-labs.org/introduction/di\\_in\\_python.html](https://python-dependency-injector.ets-labs.org/introduction/di_in_python.html) [Accessed: June 10 2026].

### Use of Digital Tools

This document has been produced exclusively for educational purposes. Referenced content, copyrights, and trademarks, remain the sole property of the respective owner.

## **Task 'Unit 11 Dependency Injection and Inversion of Control (IoC)' Unit 11**

ChatGPT 5.5 was used for some proofreading and clarity as well as literature exploration, language refinement, coding and programming techniques research. All academic writing, analysis, argumentation, and conclusions represent the original work of the author. AI tools were used for exploration, same way as academic literature exploration. Any use of content mentioned in this document is properly referenced to its original author and was used responsibly to ensure integrity and originality. I have used Artificial Intelligence responsibly and ethically, in accordance with the assignment brief. All AI use has been declared and evidenced.